DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ABES ENGINEERING COLLEGE

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Knowledge-Driven IoT Fault Diagnosis Assistant
Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:
1. Akansha Kukshal (Roll No: 2400320100117)

Degree: Bachelor of Technology (B.Tech)
Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:
Prof. / Dr. Raveendra Reddy

Department of Computer Science & Engg.

Status: 3rd Year Project Specification
Date of Submission: 30-10-2026

Github link: https://github.com/aknsh78/CSE11-117-faultdiagnosis

Document [Grab your reader's attention with a great quote from the document or use this space to emphasize a key point. To place this text box anywhere on the page, just drag it.]

Revision & Approval History

0.1	15-Sep-2026	Initial Draft & Scope Definition	Project Team	Project Guide
1.0	30-Sep-2026	Final 3rd Year Mid-Term SRS Baseline	Project Team	HOD / Committee

> **Guidance Note:** This Software Requirements Specification document conforms to the IEEE 830-1998 standard recommended practice. All sections are strictly designed for 3rd-year university lab evaluation, project reviews, and system design baselines.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, and constraints of the proposed system: **'Knowledge-Driven IoT Fault Diagnosis Assistant'**. This specification serves as a binding reference agreement between student developers, project evaluators, academic supervisors, and prospective end users during the 3rd-year engineering curriculum.

1.2 Scope of the System

The proposed application is designed to solve difficulty in real-time equipment condition monitoring and early fault diagnosis using IoT sensor data by delivering an end-to-end automated platform. Key boundary guidelines include:

- **In-Scope Capabilities:** IoT data collection, MQTT-based communication, fault diagnosis, PostgreSQL data storage, and dashboard visualization.
- **Out-of-Scope Boundaries:** Physical equipment repair, hardware manufacturing, and direct control of industrial machinery.
- **Expected Benefits:** Faster fault diagnosis, reduced manual monitoring, centralized sensor data, and improved equipment condition monitoring.

1.3 Definitions, Acronyms, and Abbreviations

Standard	**SRS**	Software Requirements Specification (IEEE Std 830-1998 format).

Architecture	**API**	Application Programming Interface for data interchange via JSON/HTTPS.
Security	**JWT / RBAC**	JSON Web Token for stateless authorization; Role-Based Access Control.
Database	**CRUD**	Create, Read, Update, Delete basic data manipulation operations.
Network	**REST**	Representational State Transfer stateless software architectural style.
Project Specific	**MQTT**	Message Queuing Telemetry Transport, used for sensor-data communication.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- **Relevant Research Papers & APIs:** MQTT documentation, PostgreSQL documentation, Python documentation, and scikit-learn documentation.

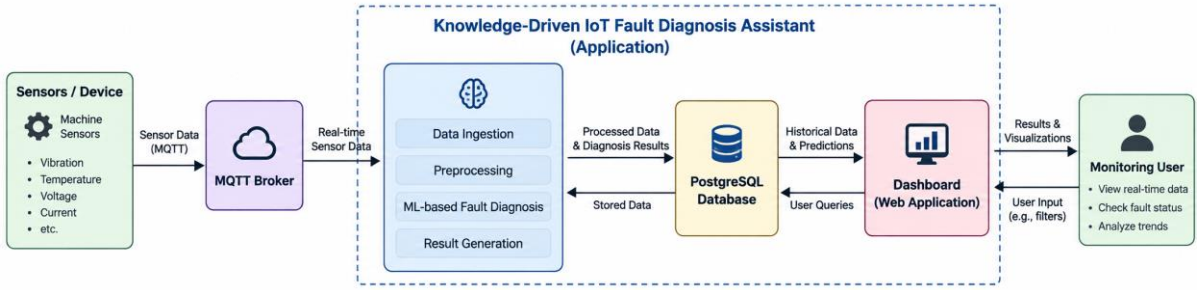
1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the product perspective, target user personas, operational context, and design constraints. Section 3 itemizes detailed functional requirements organized into logical software modules. Section 4 specifies external interfaces (UI, Hardware, Software, Network). Section 5 defines non-functional attributes (Performance, Reliability, Security). Finally, Section 6 provides standard academic modeling diagrams.

2. Overall Description

2.1 Product Perspective

The Knowledge-Driven IoT Fault Diagnosis Assistant is a self-contained system that collects sensor data, processes it for fault diagnosis, stores the data in PostgreSQL, and presents the diagnosis results through a dashboard.storage or messaging gateways) strictly through secure, authenticated RESTful endpoints.2



2.2 User Classes and Characteristics

The target software accommodates distinct user personas with varying access hierarchies:

Monitoring User	Basic / Intermediate	View sensor data, trends, fault status, and diagnosis results.
System Developer / Maintainer	Intermediate / Advanced	Maintain the IoT communication, database, diagnosis model, and dashboard.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- **Client Requirements:** Modern web browser (Google Chrome >= v110, Mozilla Firefox >= v108, Edge, Safari); minimum display resolution 1366x768 for desktop, responsive view down to 360x640.
- **Server Runtime Environment:** Linux (Ubuntu 22.04 LTS / Debian 12) or Windows Server; Python 3.x with MQTT communication and machine learning libraries.
- **Database Storage Tier:** PostgreSQL 15 / MySQL 8.0 / MongoDB 6.0 Enterprise or community editions.

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** The system should primarily use open-source software, freely available libraries, and locally available resources to minimize development and deployment costs.

- **Security Compliance:** Database credentials, MQTT credentials, and other sensitive configuration details should be protected and should not be directly exposed in the user interface or publicly accessible source code.
- **Modularity Standards:** The system should maintain a modular structure by separating IoT data acquisition, MQTT communication, data processing, fault diagnosis, database storage, and dashboard visualization components.

2.5 Assumptions and Dependencies

- **Assumptions:** The system assumes that sensor or simulated sensor data is available in the required format, network connectivity is available for MQTT communication, and the PostgreSQL database is accessible to the application.
- **Dependencies:** The system depends on the availability of the MQTT broker, PostgreSQL database, Python runtime and required libraries, machine-learning components, and the Streamlit dashboard environment.

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: IoT Data Acquisition and Communication

FR-1.1	Sensor Data Collection	Valid sensor values	High
FR-1.2	MQTT Data Transmission	Valid MQTT topic and message	High
FR-1.3	Data Reception	Valid incoming message	High
FR-1.4	Data Validation	Required fields and valid values	High

3.2 Module 2: Data Processing and Fault Diagnosis

FR-2.1	Sensor Data Processing and Storage	Valid processed sensor data	High
FR-2.2	Fault Diagnosis / ML Analysis	Valid sensor parameters	High
FR-2.3	Diagnosis Result Storage	Valid diagnosis output	High

3.3 Module 3: Reporting, Visualization & Administrative Control

FR-3.1	Dashboard Visualization	Valid sensor and diagnosis data	High

FR-3.2	Trend Analysis	Valid historical sensor data	Medium
FR-3.3	Fault Status and Recommendation	Valid diagnosis output	High

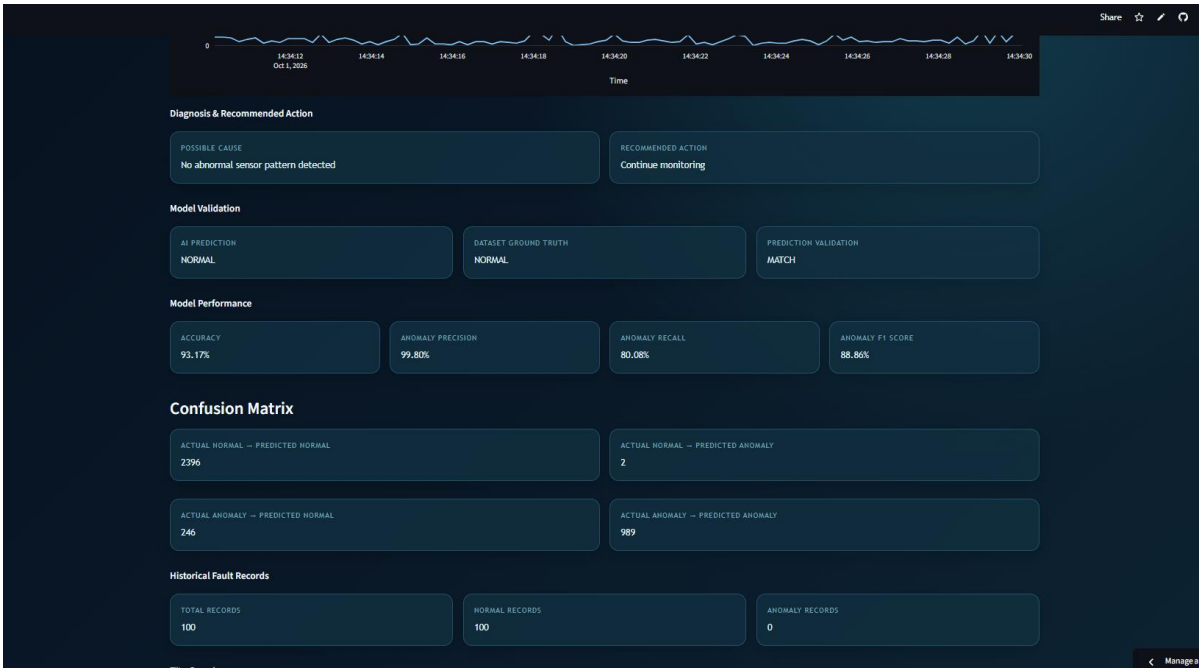
4. External Interface Requirements

4.1 User Interfaces (UI)

The system shall provide a Streamlit-based web dashboard for displaying sensor readings, trends, fault status, diagnosis results, and recommendations. The interface should provide clear navigation and readable visualizations for monitoring users.


FaultGuard AI
Predictive Maintenance System
Machine
Device: ESP32_001
Status: Connected
Data Source
SKAB Valve Dataset
Random Forest
MQTT + HiveMQ
PostgreSQL
Refresh Dashboard
FaultGuard AI
Intelligent Industrial Fault Detection & Predictive Maintenance
LIVE MONITORING
MACHINE HEALTH STATUS
SYSTEM NORMAL
No abnormal sensor pattern detected.
Key Machine Indicators
TEMPERATURE
68.76 °C
CURRENT
1.08 A
VOLTAGE
211.60 V
FLOW RATE
31.98
Sensor Monitoring
ACCELEROMETER 1 RMS
0.03
CURRENT
1.08
ACCELEROMETER 2 RMS
0.04
PRESSURE
0.05
TEMPERATURE
68.76
VOLTAGE
211.60
THERMOCOUPLE
24.84
VOLUME FLOW RATE RMS
31.98


FaultGuard AI
Predictive Maintenance System
Machine
Device: ESP32_001
Status: Connected
Data Source
SKAB Valve Dataset
Random Forest
MQTT + HiveMQ
PostgreSQL
Refresh Dashboard
Temperature
Temperature (°C)
Time
Current
Current (A)
Time
Voltage
Voltage (V)
Time
Vibration
RMS
Accelerometer 1 RMS
Accelerometer 2 RMS
Manage app



4.2 Hardware Interfaces

Specification of physical interaction points:

- **Standard Deployment:** Keyboard, mouse, and display for accessing the monitoring dashboard.
- **Embedded / IoT Variants (if applicable):** ESP32-based sensor interface for collecting machine parameters such as temperature, vibration, voltage, current, pressure, and flow rate.

4.3 Software Interfaces

- **Relational Database Connector:** PostgreSQL database for storing sensor readings and fault diagnosis results.
- **E:** Amazon Web Services (S3 bucket for document storage) / Cloudinary for image optimization.

4.4 Communication Interfaces

- **IoT Communication Protocol:** MQTT for transmitting sensor data between the data source and diagnosis system.
- **Dashboard Communication:** HTTP/HTTPS communication for accessing the Streamlit-based web dashboard.

5. Non-Functional Requirements (NFRs)

5.1 Performance	Sensor data processing and dashboard updates should occur with minimal delay during normal operation.	Test data processing and dashboard refresh response..
5.2 Security	Database credentials, MQTT credentials, and other sensitive configuration details should not be exposed publicly.	Review configuration files and application source code.
5.3 Reliability	The system should continuously receive, process, and store valid sensor data during normal operation.	Test MQTT communication, database storage, and system recovery.
5.4 Portability	Dashboard should work on modern browsers such as Chrome, Edge, and Firefox and support desktop displays.	Test dashboard across supported browsers and screen sizes.

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application:

Use Case Diagram	Maps the Monitoring User and System Developer/Maintainer to the main system interactions.	UML Standard diagram + Use Case specification tables.
Data Flow Diagram (DFD)	Illustrates the flow of sensor data through MQTT, fault diagnosis, PostgreSQL, and the dashboard.	DFD Level 0 and Level 1
Entity-Relationship (ERD)	Shows the database entities, attributes, and relationships for sensor and diagnosis records.	ER Diagram / Crow’s Foot notation
Activity / Sequence Diagram	Shows the chronological flow of sensor data from collection through diagnosis, storage, and dashboard display.	UML Activity or Sequence Diagram

6.1 Use Case Diagram

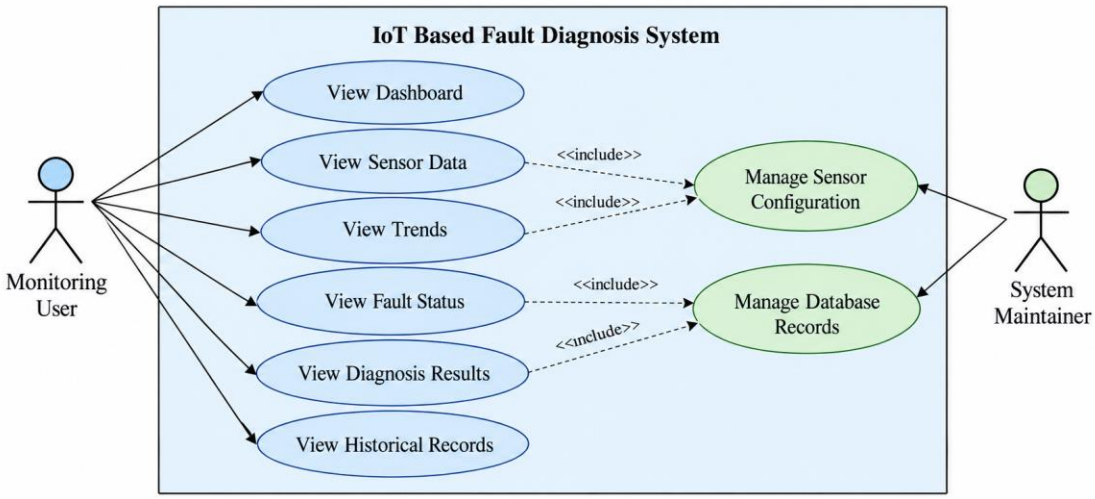


Figure 6.1: Use Case Diagram

6.2 Data Flow Diagram (DFD)

Level 0 DFD

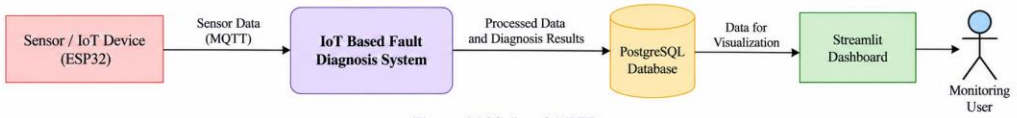


Figure 6.2(a): Level 0 DFD

Level 1 DFD

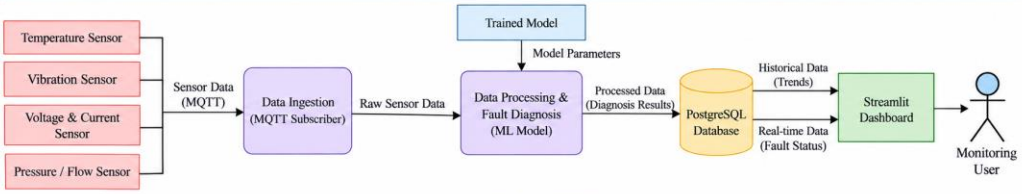


Figure 6.2(b): Level 1 DFD

6.3 Entity-Relationship Diagram (ERD)

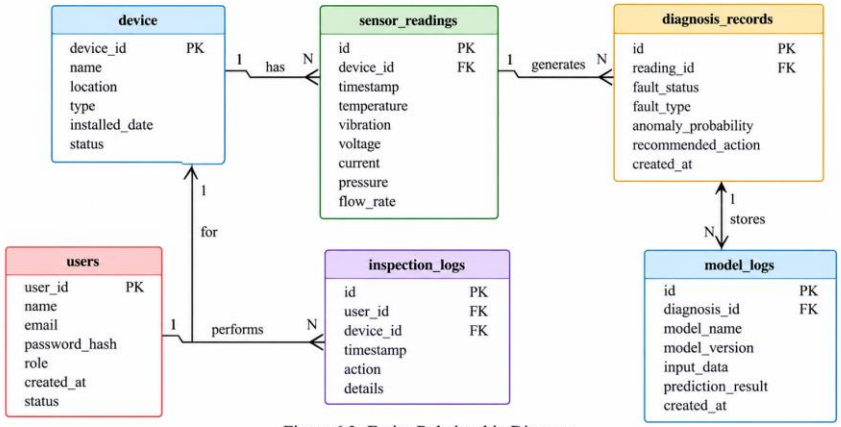


Figure 6.3: Entity-Relationship Diagram

6.4 Sequence Diagram

The sequence diagram shows the interaction between components for real-time fault diagnosis and visualization.

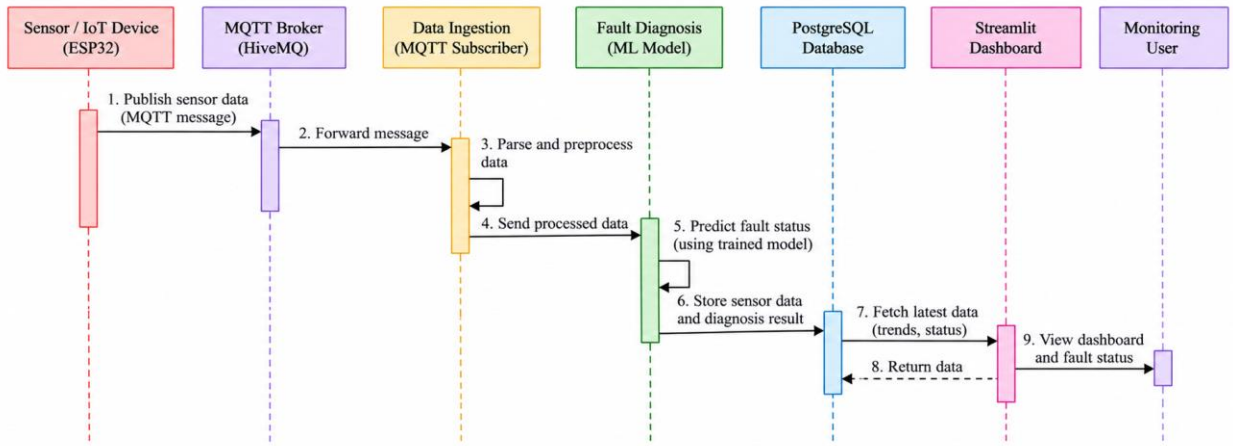


Figure 6.4: Sequence Diagram for Fault Diagnosis System

7. Academic Review & Sign-Off

This Software Requirements Specification document has been submitted and reviewed by the academic supervisory panel:

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____